Customer Shopping Behaviour Analysis

Data cleaning, SQL analysis and a Power BI dashboard built on 3,900 purchase records

Project report

Tools: Python (pandas, matplotlib), PostgreSQL, Power BI, Git and GitHub

Data: customer_shopping_behavior.csv (3,900 purchase records)

November 2026

Contents

1. Introduction
2. Objectives
3. The dataset
4. Tools and workflow
5. Data cleaning
6. Analysis and findings
7. Power BI dashboard
8. Recommendations
9. Code review before upload
10. Limitations
11. Next steps
12. How to run the project
13. Conclusion

Appendix A. Repository structure

1. Introduction

Most shops collect far more information about their customers than they ever look at. A spreadsheet of purchases can show what sold, but it rarely says why, or which customers are worth paying attention to. This project starts from one such file, 3,900 purchase records with details about the customer, the item, the shipping and the discount, and asks a handful of practical questions about it.

I wanted the project to look like a small analytics job rather than a single notebook. Python cleans the data, PostgreSQL answers specific business questions, and Power BI brings the results together on one page where someone can filter by gender, category, subscription status and shipping type. I also kept everything in a form that someone else could open, read and run.

The rest of the report covers the data, the cleaning decisions, the answers to the ten questions, what the dashboard shows, and a review of the code I went through before publishing it. That review turned up several real bugs, which are described in Section 9.

2. Objectives

- Clean the raw file and record every change, so that each step can be checked.
- Answer ten business questions with SQL and explain what each answer means.
- Build a dashboard that lets someone explore the results without touching the code.
- Check the code for errors and make the project easy to reproduce on another machine.

3. The dataset

The raw file has 3,900 rows and 18 columns. Every customer ID is unique, so each row is one purchase by one customer. The file has no order date, which means the analysis cannot show how things change over time. The only time-related field is season, and that limitation shapes several conclusions later in the report. The 25 products belong to four categories: Clothing (11 products), Accessories (8), Footwear (4) and Outerwear (2).

Table 1. Columns in the cleaned table

Column	Type	What it holds
customer_id	Integer	Unique customer identifier
age	Integer	Customer age, from 18 to 70
gender	Text	Male or Female
item_purchased	Text	Product name (25 different products)
category	Text	Clothing, Accessories, Footwear or Outerwear
purchase_amount	Integer	Amount paid in US dollars, from 20 to 100
location	Text	US state of the customer
size	Text	Clothing size
color	Text	Colour of the product
season	Text	Season of the purchase
review_rating	Decimal	Customer rating from 2.5 to 5.0 (blanks filled in cleaning)
subscription_status	Text	Yes or No
shipping_type	Text	One of six shipping options

Column	Type	What it holds
discount_applied	Text	Yes or No
previous_purchases	Integer	Number of earlier purchases, from 1 to 50
payment_method	Text	One of six payment methods
frequency_of_purchases	Text	How often the customer buys (seven labels)
age_group	Text	Quartile group added during cleaning
purchase_frequency_days	Integer	Approximate days between purchases, added during cleaning

4. Tools and workflow

The project has three stages, shown in Figure 1. The first is a Python script that reads the raw file, cleans it and saves a cleaned copy. That copy is the only input for the other two stages, the SQL queries in PostgreSQL and the Power BI dashboard. Keeping one cleaned file in the middle means the SQL answers and the dashboard numbers always come from the same data.

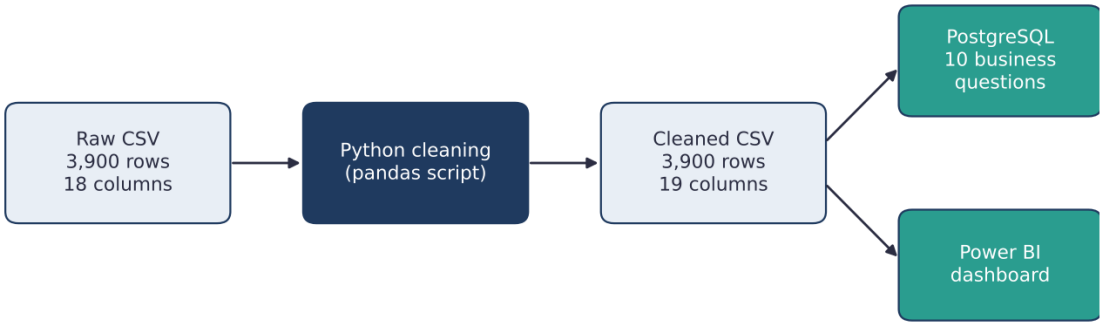


Figure 1. Project workflow. Python produces the cleaned file that feeds both the SQL queries and the Power BI dashboard.

The tools are Python 3 with pandas and matplotlib, PostgreSQL for the queries, Power BI Desktop for the dashboard file (.pbix), and Git and GitHub for version control.

5. Data cleaning

Cleaning was mostly routine, but a few decisions are worth writing down. Table 2 lists each step. After cleaning, the file has 3,900 rows, 19 columns and no missing values.

Table 2. Cleaning steps

Step	What I did	Why
1	Read the file as UTF-8 with the hidden byte-order mark removed.	The file starts with an invisible marker that can stick to the first column name.
2	Filled the 37 missing review ratings (about 1%) with the median rating of the same category.	The median is not pulled around by a few extreme ratings. If a whole category were blank, the overall median would be used.
3	Renamed the columns to lowercase snake_case. For example, Purchase Amount (USD) became purchase_amount.	Names are easier to write in both SQL and Python.
4	Added age_group from the quartiles of age.	Gives four groups of similar size. These are analysis groups, not official age bands (see Section 10).

Step	What I did	Why
5	Converted purchase frequency into approximate days, for example Weekly to 7 and Monthly to 30.	Lets frequency be used as a number. An unknown label now stops the script.
6	Dropped promo_code_used.	It matches discount_applied in all 3,900 rows, so keeping both would count every discount twice.
7	Checked the result for missing values, duplicate IDs and out-of-range ages, amounts and ratings.	A simple guard against silent errors.

6. Analysis and findings

Each question below comes from the SQL file. I give the answer first and then what I think it means. Money is in US dollars. Table 3 gives the headline numbers.

Table 3. Key figures for the whole dataset

Measure	Value
Purchase records (one per customer ID)	3,900
Total revenue	$233,081
Average purchase amount	$59.76
Median purchase amount	$60
Customers with a subscription	27.0%
Purchases that used a discount	43.0%
Average review rating	3.75 out of 5

6.1 Revenue by gender and category (Q1)

Men account for 2,652 purchases and $157,890 of revenue, which is 67.7% of the total. Women account for 1,248 purchases and $75,191, or 32.3%. The gap comes from how many purchases each group made, not from how much each purchase cost. The average purchase is $60.25 for women and $59.54 for men, which is almost identical. Figure 2 shows the split.

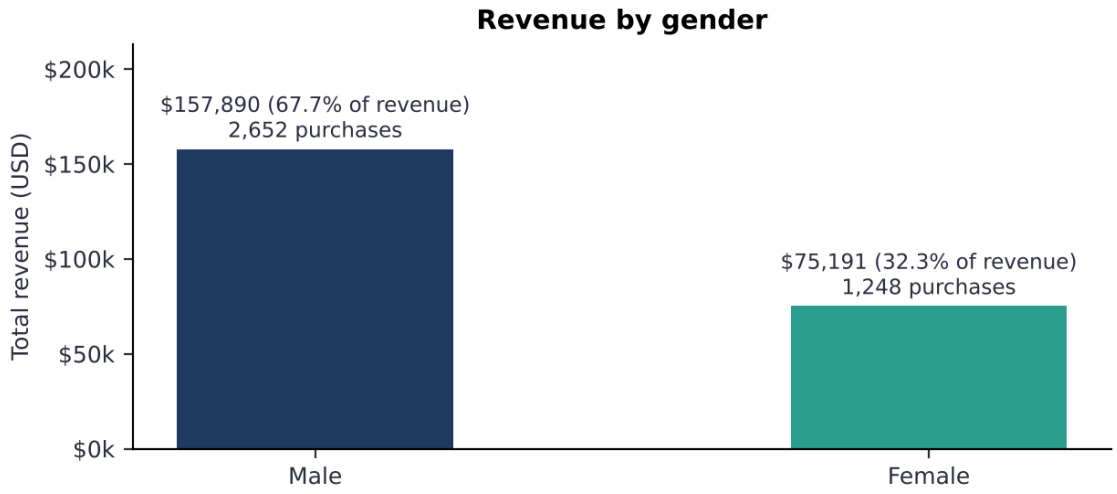


Figure 2. Total revenue by gender. Men make up 68% of purchases and 67.7% of revenue.

By category, Clothing brings in the most revenue at $104,264 (44.7%), followed by Accessories at $74,200 (31.8%), Footwear at $36,093 (15.5%) and Outerwear at $18,524 (8.0%). Figure 3 shows the same picture. Outerwear is small because it has only two products, and each one sells about as often

as the products in the other categories. Taken together, the gender and category results suggest that growing the number of customers is a bigger lever here than raising what each customer spends.

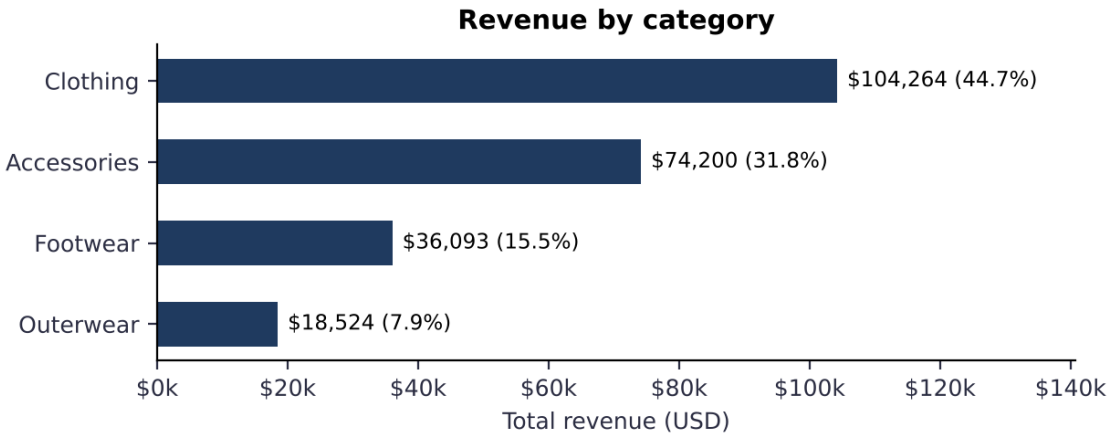


Figure 3. Revenue by category. Outerwear is the smallest category, with only two products.

6.2 Revenue by age group (Q10)

Revenue is spread fairly evenly across the four age groups. Young Adults bring in the most, $62,143 or 26.7% of revenue, and the Senior group brings in the least, $55,763 or 23.9%. The groups are quartiles, so each holds roughly a quarter of the customers, which is why the shares sit so close together. The Senior group covers ages 58 to 70, which is a wide range for that label, and Section 10 explains why I kept it for now.

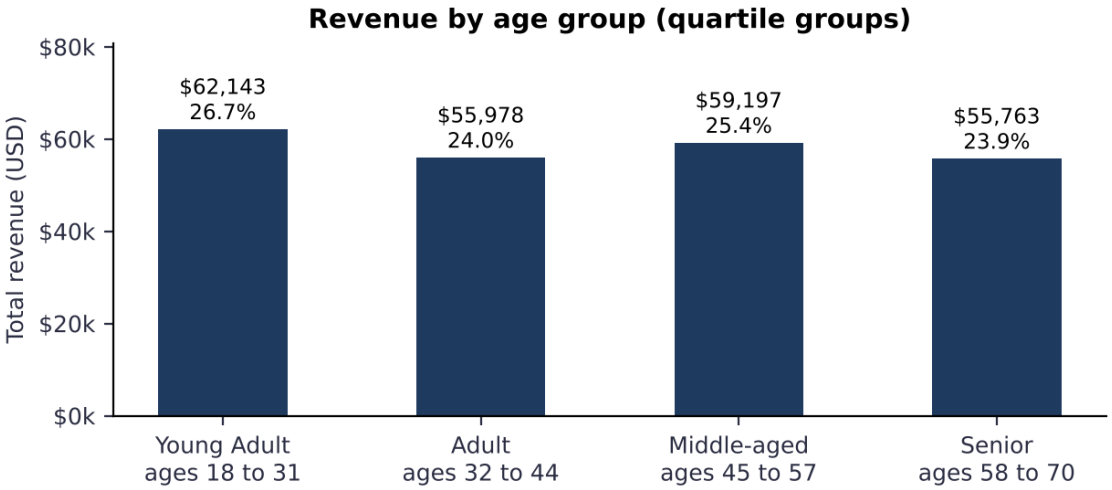


Figure 4. Revenue by age group. The groups are quartiles, so each holds about a quarter of customers.

6.3 Product ratings and best sellers (Q3 and Q8)

The five products with the highest average rating are Gloves (3.86), Sandals (3.84), Boots (3.82), Hat (3.80) and Skirt (3.78). These five are only a few hundredths apart, and all 25 products sit between 3.62 and 3.86 (Figure 5). Nothing in this data stands out clearly, so I would not change stock or prices based on this ranking alone.

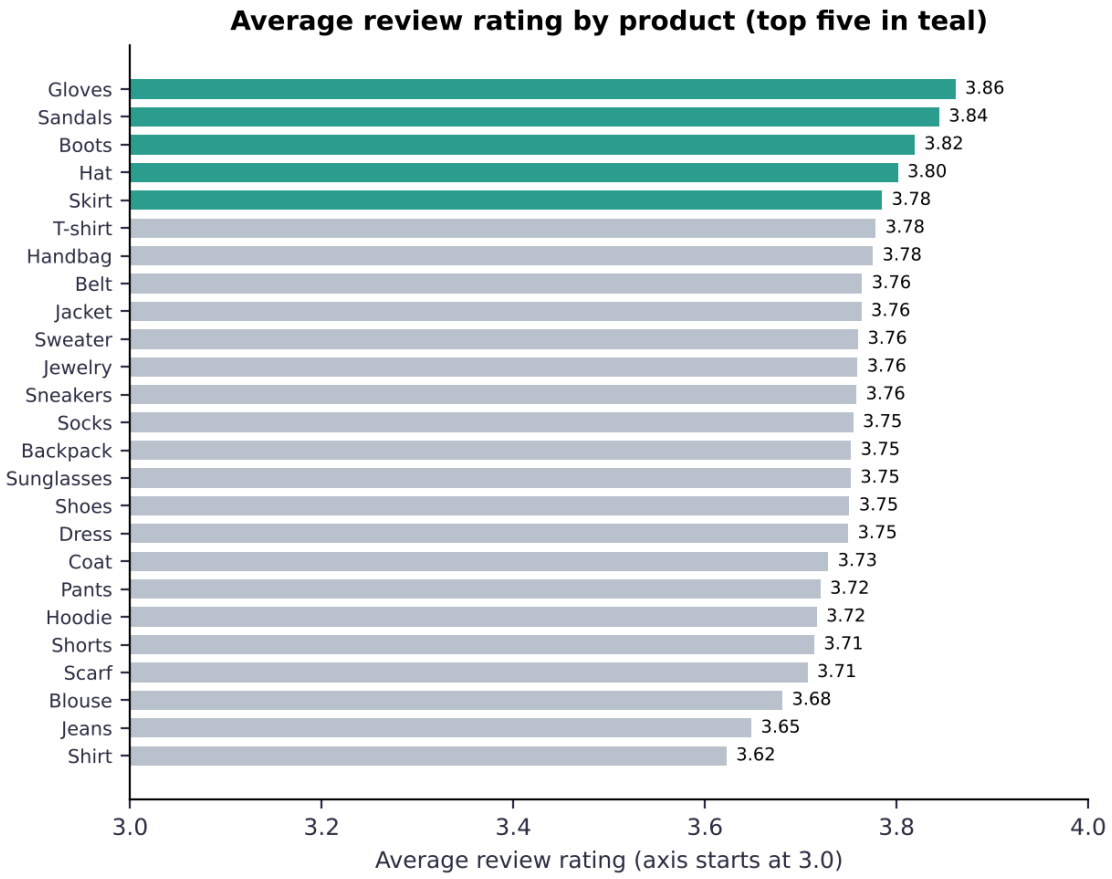


Figure 5. Average review rating for each product (Q3). Teal bars are the top five. The axis starts at 3.0 so that small differences are visible.

For Q8, I ranked products by how often they were bought within each category, giving tied products the same rank. Table 4 shows the top three in each category. Pants and Blouse tie at the top of Clothing, and Sunglasses and Belt tie for second in Accessories. Outerwear has only two products, so it has only two rows.

Table 4. Top three products by purchases in each category (Q8)

Category	Rank	Product	Purchases
Accessories	1	Jewelry	171
Accessories	2	Belt	161
Accessories	2	Sunglasses	161
Clothing	1	Blouse	171
Clothing	1	Pants	171
Clothing	3	Shirt	169
Footwear	1	Sandals	160
Footwear	2	Shoes	150
Footwear	3	Sneakers	145
Outerwear	1	Jacket	163
Outerwear	2	Coat	161

6.4 Discounts and shipping (Q2, Q4 and Q6)

Discounts were used on 43% of purchases (1,677 of 3,900). They are spread across the product range rather than concentrated in a few items, with rates from about 33% to 50% by product. Q6 lists the five highest: Hat at 50.0%, Sneakers at 49.7%, Coat at 49.1%, Sweater at 48.2% and Pants at 47.4% (Figure 6).

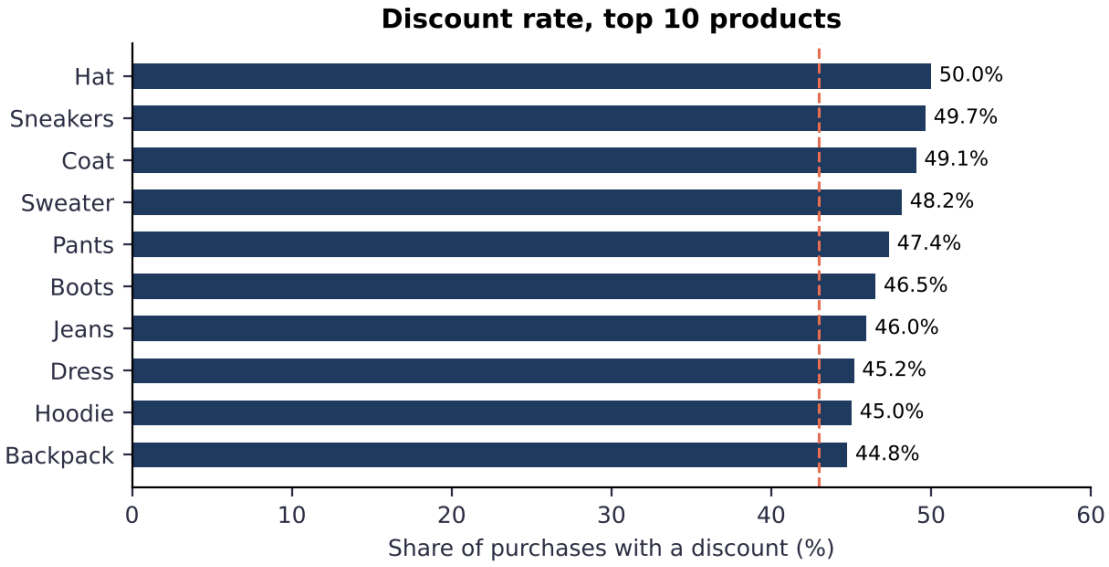


Figure 6. Share of purchases with a discount, top 10 products (Q6). The dashed line marks the overall rate of 43%.

Q2 asks which discounted purchases were above the average amount. The answer is 839 purchases, almost exactly half of the 1,677 discounted ones. If discounts were pulling larger orders in, that share would be much higher. The discount does not seem to change how much people spend: discounted purchases average $59.28, and full-price purchases average $60.13.

Q4 compares Standard and Express shipping. Express orders average $60.48 and Standard orders $58.46, a gap of about $2. Figure 7 shows all six shipping types. They all fall between $58.46 and $60.73, so the shipping option does not appear to change how much people spend. I would choose shipping on cost and delivery speed rather than on order value.

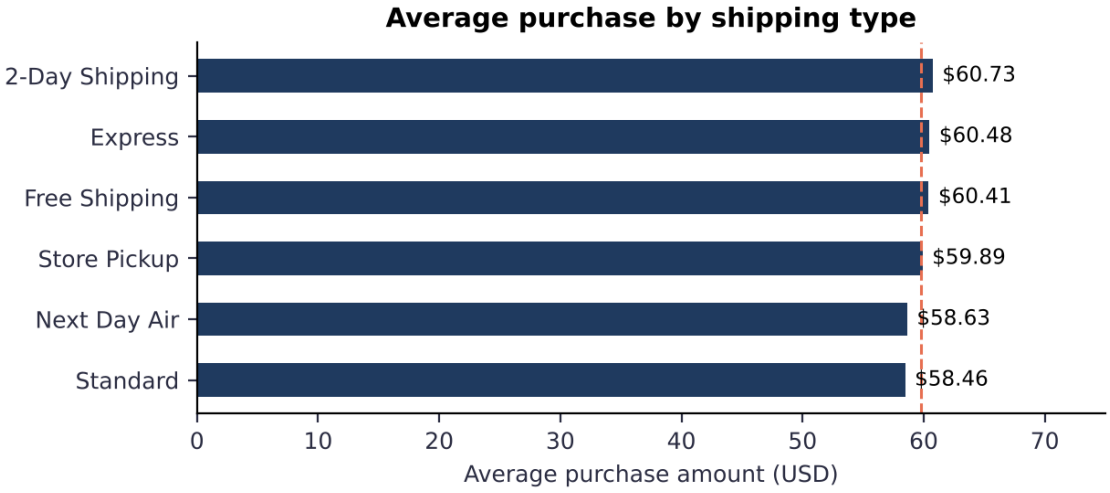


Figure 7. Average purchase by shipping type. The dashed line is the overall average of $59.76, and all six options sit within about $2.30 of each other.

6.5 Subscriptions and loyalty (Q5, Q7 and Q9)

About 27% of customers (1,053 people) have a subscription. Subscribers spend an average of $59.49 per order, and non-subscribers spend $59.87. Non-subscribers bring in far more total revenue, $170,436 compared with $62,645, but that mostly reflects their larger numbers. In this data, subscribers do not spend more per order (Figure 8).

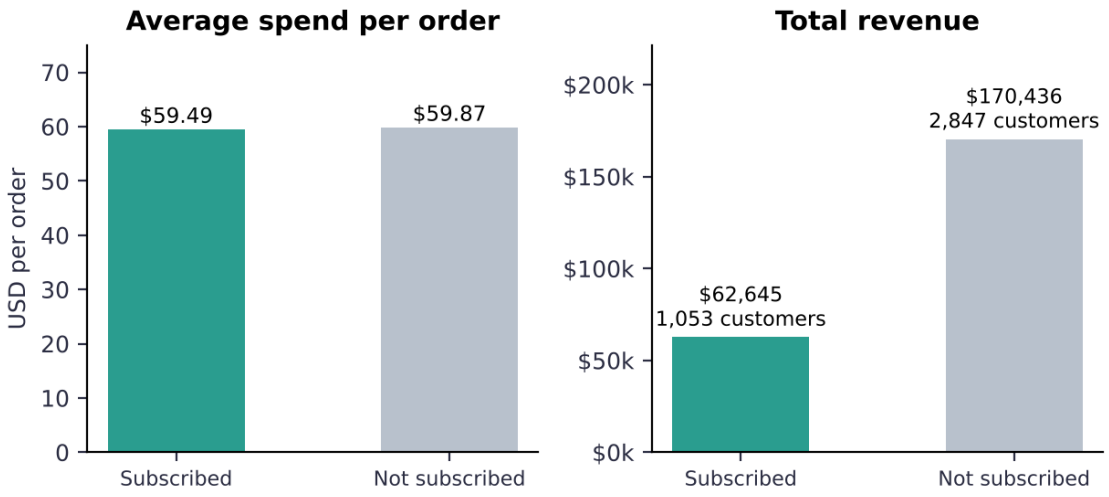


Figure 8. Average spend per order and total revenue for subscribers and non-subscribers (Q5).

The loyalty segments in Q7 put 79.9% of customers in Loyal, 18.0% in Returning and only 2.1% in New. The problem is the thresholds. Figure 9 shows that previous purchases are spread almost evenly from 1 to 50, so a cut-off at 11 leaves most people in one large group. The quartile points for this data are 13, 25 and 38. Splitting customers at those points would give four groups that are easier to act on.

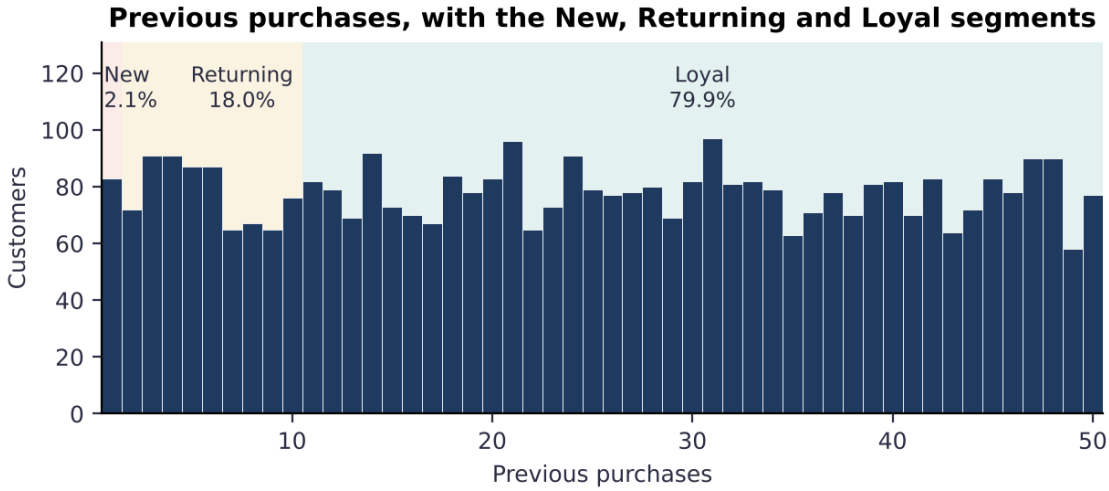


Figure 9. Number of previous purchases. The shading marks the New, Returning and Loyal segments from Q7.

Repeat buyers, meaning customers with more than five previous purchases, subscribe at 27.6%. Everyone else subscribes at 22.4% (Figure 10). The gap is about 5 percentage points, and the 95% range around it runs from about 1 to 9 points, with a two-sided p-value of about 0.02. This is the clearest behavioural difference I found. The comparison group is small, at 424 customers, so I would treat it as a reason to test a subscription offer rather than as settled proof. The original Q9 query only counted repeat buyers, which says how many there are but not whether they are more likely to subscribe, so I changed it to compare rates.

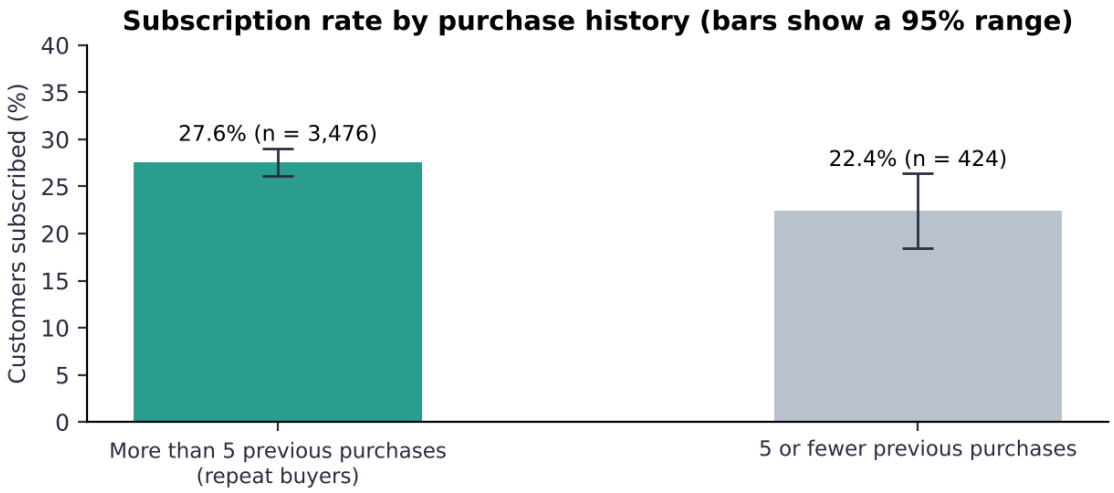


Figure 10. Subscription rate for repeat buyers and other customers (Q9). The bars show a 95% range.

6.6 Review ratings

Ratings average 3.75 with a median of 3.8. About 65% of ratings fall between 3.0 and 4.5, and the standard deviation is 0.71 (Figure 11). The spread across individual ratings is much wider than the spread between product averages, which is another sign that ratings do not separate the products much.

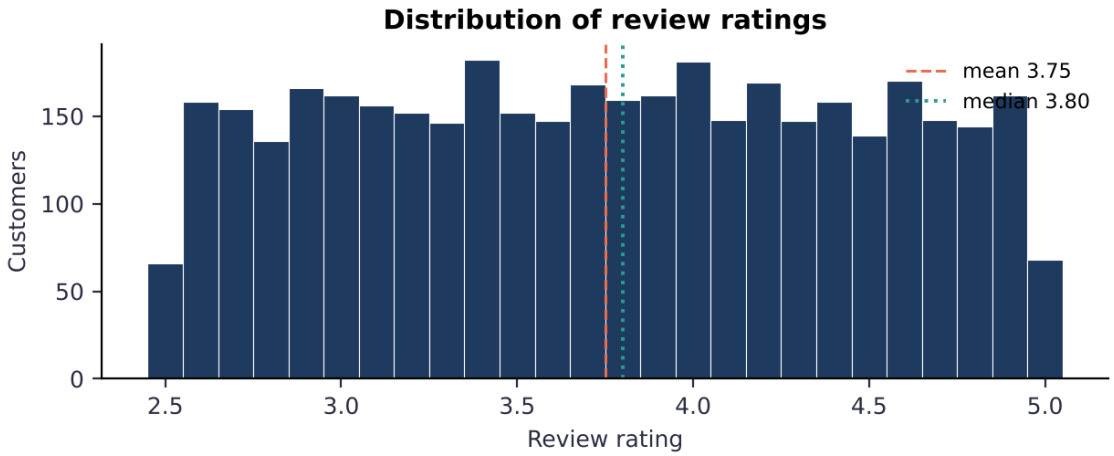


Figure 11. Distribution of review ratings. The mean and median are close together.

7. Power BI dashboard

The dashboard is a single report page sized for 1920 by 1080 pixels. It has 18 visual objects, ten of which show data and eight of which are shapes or text boxes used for layout. Table 5 lists the data visuals. The dashboard reads the cleaned CSV, so it matches the SQL answers when both use the same file.

Table 5. Data visuals on the dashboard page

Visual	What it shows	Notes
KPI card	Number of customers, average purchase amount, average review rating	Uses three named measures, which are defined in the Power BI model

Visual	What it shows	Notes
Donut chart	Count of customers by gender	The title mentions discount applied, but the chart does not use it, so the title is out of date
Column chart, Revenue By Category	Sum of purchase amount by category	Works as written
Column chart, Sales By Category	Uses a count, not a sum, on purchase amount	Probably shows the number of purchases under a sales label. Please check
Bar chart, Revenue By Age Group	Sum of purchase amount by age group	This visual appears twice. Remove one copy
Four slicers	Subscription status (two), gender with subscription, category, shipping type	Filter the whole page

Before the dashboard is shared, four things need attention. The Sales By Category chart appears to count purchases rather than add up sales, so either its title or its calculation should change. The donut chart title mentions discounts that the chart does not use. The age group chart appears twice. The DAX measures behind the KPI card should be written down somewhere, because they are stored inside the model and cannot be read easily from the file. I could only inspect the file's structure, not open it in Power BI Desktop, so please open it once yourself to confirm these points.

8. Recommendations

These come from the findings above. Each one is a suggestion to test, not a proven result.

1. **Test a subscription invitation for repeat buyers.** Repeat buyers subscribe more often. A simple test would offer a subscription after a customer's fifth purchase, then compare subscription rates against a similar group that does not receive the offer.
2. **Be careful with broad discounts.** Discounts are used on about 43% of orders, and discounted orders are not larger. If discounts cost margin, smaller or better-targeted offers may reach the same customers at lower cost. I do not have margin data, so this is a question to check, not a conclusion.
3. **Redraw the loyalty tiers.** Use the quartile points of 13, 25 and 38, or another split that puts a meaningful share of customers in each tier.
4. **Choose shipping on cost and speed.** Shipping type does not change average order value in any meaningful way, so it should be decided as a cost and service question.
5. **Collect order dates and costs.** Without dates the analysis cannot show trends, and without costs it cannot show profit. These two fields would do more for the next version of this project than another chart.

9. Code review before upload

Before uploading, I ran every part of the project: the Python script from start to finish, each SQL statement, and the structure of the dashboard file. The cleaning logic was sound, but the script stopped at the very end, and the SQL file would not run as one script because three queries had no semicolon. Table 6 lists what I found and what changed. The fixed versions are in the GitHub folder.

Table 6. Problems found and changes made

#	Area	Problem found	Change made
1	Python	Stopped with ModuleNotFoundError, because sqlalchemy was imported near the end of the script and is not installed by default.	The import moved into the optional PostgreSQL step. requirements.txt lists the packages.
2	Python	The cleaned CSV was never saved, but the SQL and the dashboard both need it.	The script now writes data/customer_shopping_cleaned.csv.
3	Python	File names were relative, so the script only worked from its own folder.	Paths are built from the location of the script.
4	Python	The password was a placeholder in the code, with a comment saying never to hardcode it, and a DB_LOAD switch that did not exist.	The password comes from the PG_PASSWORD environment variable. Loading only happens with --load-postgres.
5	Python	Unknown purchase frequencies printed a warning and left blanks in the data.	The script now stops with an error.
6	Python	Age group was saved as a category type, which some database tools handle poorly.	It is saved as plain text.
7	Python	Missing ratings would stay blank if a whole category were empty. An import was unused and step numbers skipped from 9 to 11.	Falls back to the overall median. Unused code removed, and validation checks added.
8	SQL	Q1, Q2 and Q3 had no semicolon, so the file fails when run as one script.	Semicolons added. The whole file now runs statement by statement.
9	SQL	The COPY command used a fixed Windows path and needs the server to read the file.	Replaced with \copy and a relative path, with pgAdmin and Python options as alternatives.
10	SQL	Q5 sorted total revenue from lowest to highest, because DESC only applied to average spend.	ORDER BY total_revenue DESC.
11	SQL	Q9 returned counts, which depend on how large each group is.	Returns the subscription rate for each group instead.
12	SQL	Q8 used ROW_NUMBER, which breaks ties arbitrarily. Equal products (Pants and Blouse, Sunglasses and Belt) looked as if one was ahead of the other.	Changed to RANK, so tied products share a rank.
13	SQL	Q2 used >= where the question says more than.	Changed to >. The result does not change here, because no purchase equals the average exactly.
14	SQL	The ::numeric cast only works in PostgreSQL.	Replaced with the standard CAST, and the query no longer needs it.
15	SQL	Some results had no ORDER BY, so row order was not stable between runs.	Added ordering and clearer column names.
16	Power BI	Sales By Category appears to count purchases instead of summing sales.	Flagged for checking in Power BI Desktop (Section 7).
17	Power BI	Revenue By Age Group appears twice.	Flagged. Remove one copy.
18	Power BI	The DAX measures are not documented anywhere.	Flagged. Definitions should be added to the README.
19	Power BI	The donut chart title mentions discount applied, but the chart only uses gender.	Flagged. Rename the title, or add discount_applied to the chart.

Something I kept on purpose. The age group labels come from quartiles, so 'Senior' covers ages 58 to 70. I kept the labels so that the SQL file and the dashboard stay consistent. A future version should rename them or use fixed age bands.

How I tested. The cleaning script ran from start to finish and reproduced the cleaned file exactly. The SQL file ran statement by statement with no errors, and its results matched a separate pandas calculation. PostgreSQL was not available in my environment, so I ran the queries in SQLite, which handles the same statements here. Please run the PostgreSQL version once on your own machine. The dashboard was checked by reading the structure of its file, not by opening it in Power BI Desktop.

10. Limitations

- **There is no purchase history.** Each customer ID appears once, so 'repeat buyer' means a count of earlier purchases, not an observed sequence of orders.
- **The data may not be real.** Purchase amounts are spread almost evenly from $20 to $100 (standard deviation $23.69), which is unusual for real retail sales. The file may be a sample or simulated data, so treat the numbers as practice results rather than business facts until they are checked against real records.
- **The age groups are quartiles, not age bands.** The Senior group covers ages 58 to 70, which is not what most people mean by the word.
- **The repeat-buyer comparison is small.** Only 424 customers have five or fewer previous purchases.
- **There are no dates.** Trends over time cannot be measured, and season is the only time-related field.
- **There are no costs.** Revenue cannot be turned into profit, so margin-based decisions need more data.
- **The dashboard was checked from its file structure.** It was not opened and tested in Power BI Desktop.

11. Next steps

- Add order dates and run a trend analysis by month and season.
- Replace the quartile age groups with fixed bands, for example 18 to 24, 25 to 34, 35 to 44, 45 to 54 and 55 and over.
- Add cost data so that the analysis can compare profit by category, shipping option and discount.
- Test the subscription offer for repeat buyers with a control group.
- Add a GitHub Actions workflow so the tests in the tests folder run automatically after every change.

12. How to run the project

These steps assume Python 3.9 or newer, and PostgreSQL and Power BI Desktop if you want to use those parts.

1. Download or clone the repository, then open a terminal in the project folder.
2. Install the packages: `pip install -r requirements.txt`
3. Clean the data: `python customer_shopping_analysis.py`. This writes data/customer_shopping_cleaned.csv.

4. Optional, PostgreSQL: create the table with the first part of customer_behavior_queries.sql, then load the CSV with `\copy` from psql, with pgAdmin's Import tool, or with `python customer_shopping_analysis.py --load-postgres` after setting PG_PASSWORD.
5. Power BI: open dashboard/Customer_Behavior_Dashboard.pbix. If it asks for the data source, point it at data/customer_shopping_cleaned.csv.
6. Rebuild this report: `python reports/make_report.py`
7. Run the checks: `python tests/test_pipeline.py`

13. Conclusion

The data tells a fairly simple story. Men and women spend about the same per order, and most of the revenue gap comes from how many purchases each group makes. Discounts and shipping choices barely move order size. Subscribers do not spend more per order, although repeat buyers are somewhat more likely to subscribe, and that is the most useful lead in the project.

The next step is better data rather than more charts: real order dates, costs and a clearer loyalty definition would make the findings far more useful. The code is now cleaner, reproducible, and the bugs I found are fixed.

Appendix A. Repository structure

```
Customer_Behavior_GitHub/
  README.md
  CODE_REVIEW.md
  requirements.txt
  .gitignore
  customer_shopping_analysis.py      cleaning pipeline
  customer_behavior_queries.sql      table, load steps and Q1 to Q10
  data/
    customer_shopping_behavior.csv   raw data
    customer_shopping_cleaned.csv    cleaned data (created by the script)
  dashboard/
    Customer_Behavior_Dashboard.pbix Power BI report
  reports/
    make_report.py                   builds this PDF and the charts
    Customer_Behavior_Project_Report.pdf
    figures/                         charts used in the report
  tests/
    test_pipeline.py                 basic checks for the cleaning step
```